

Field API interface for ecTrans

Haumont Denis (RMI), Marguinaud Philippe and Chesnel Jeremy (Météo-France), Deconinck Willem and Hatfield Sam (ECMWF)



INTRODUCTION

MOTIVATIONS

- Improve code flexibility
- Facilitate host to device synchronization
- Finalize the replacement of PGP/PSP arrays by Field API in ARPEGE/IFS

CODE FLEXIBILTY

Current interface: DIR_TRANS and INV_TRANS (20 years old)

```
CALL INV_TRANS(&
   & PSPVOR=PSPVOR,PSPDIV=PSPDIV,PSPSC2=ZSP(1:IST,:),PSPSC3A=PSPHV,&
   & PSPSC3B=PSPGFL,&
   & LDSCDERS=LLSCDERS,LDVORGP=LDVOR,LDDIVGP=LLDIVGP,LDUVDER=LLUVDER,&
   & KRESOL=NRESOL,KPROMA=NPROMA,KVSETUV=NBSETLEV,KVSETSC2=IVSETSC(1:1),&
   & KVSETSC3A=NBSETLEV,KVSETSC3B=NBSETLEV,&
   & PGPUV=PGMV(:,:,ISTUV:IENUV,:),&
   & PGP2=PGMVS(:,1:IDIMGMVS,:),&
   & PGP3A=PGMV(:,:,ISTGMV:IENGMV,:),&
  & PGP3B=PGFL(:,:,IOFF+1:IOFF+IDIM3,:))
```



PGP/PSP array slices

Fields transmitted using PGP/PSP array slices to inv_trans and dir_trans

PGP/PSP array slices: drawbacks (1/3)

- Lack of flexibility
  - only predefined subset of fields can be treated at the same time

  (the ones in contiguous area of PGP/PSP arrays)

  - aggregation of different function calls very difficulty
  - not possible to transform single field
  - some fields are mandatory

    (A dummy field has to be passed when not used)

PGP/PSP array slices: drawbacks (2/3)

- FSPFGL_PROC procedure is passed as argument
  Not flexible

- Slices relies on hidden assumptions : complicated code

Example: TRANSINV_MDL.F90

```
IF(NBSETSP == MYSETV) THEN
  IST = 1
  ZSP(IST,:) = PSPSP(:)
ELSE
  IST = 0
ENDIF
IF( LDVOR ) THEN
  ISTUV = 1
ELSE
  ISTUV = 2
ENDIF
IF(.NOT. LLSCDERS) THEN
  IENGMV = MAX(ISTGMV,ISTGMV-1+(IENGMV-ISTGMV+1)/3)
  IDIMGMVS = MAX(1,IDIMGMVS/3)
  ISTUV = YT0%MU
  IENUV = YT0%MV
ENDIF
IOFF = NUMFLDS-NUMSPFLDS
IDIM3 = NUMSPFLD
```

PGP/PSP array slices: drawbacks (3/3)

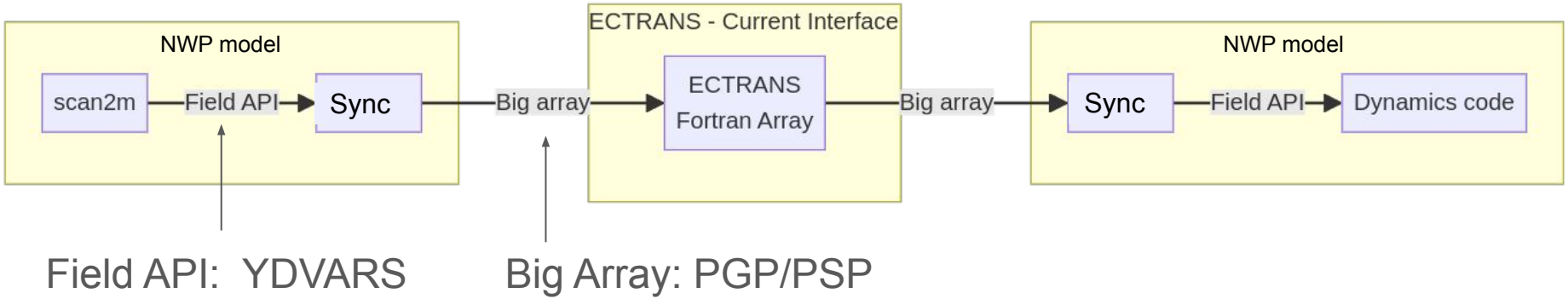
- No compatibility between PGP/PSP and YDVARS on GPU

  ⇒ Not gpu friendly

  - complicated memory management
  - unnecessary memory transfers

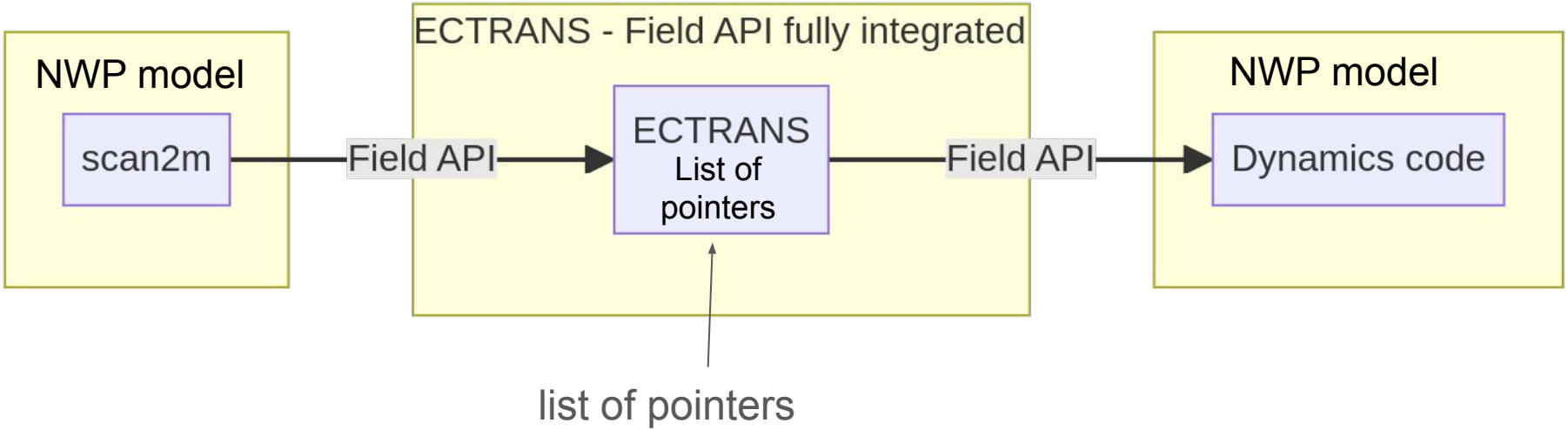
UPDATE Situation improved recently (same layout on CPU and GPU) (Field Stack)

ECTRANS INTEGRATION IN NWP - REFERENCE



Sync : on the accelerator, make sure that Field API pointers and PGP/PSP contains the same information (LEGACY_FIELD) - GPU to GPU copies

ECTRANS INTEGRATION IN NWP - NEW INTERFACE



- ecTrans refactored to use list of pointers instead of PGP/PSP
- optimal: no memory transfers

TIMELINE

- Phase 0 - Preparation work
- Phase 1 - Thin interface Field API on top of existing ecTrans
- Phase 1B (NEW) - Introduction of an intermediate Field View interface
- Phase 2 - List of pointers in ecTrans
- Phase 3 - Refactoring of NWP code

PREPARATION WORK

Phase 0



MPL & ecTrans meeting - 03.11.2026
A Field API interface for ecTrans

PREPARATION WORK

- Early prototypes - experimentation
  - **inv_trans**: first prototype demonstrating the interface
    https://github.com/dhaumont/inv_trans/
  - **Arpege integration test**
    https://github.com/dhaumont/IAL/tree/50t1_field API_ectrans/
  - **inv_trans_split**: propagating list of pointers inside ectrans
    https://github.com/dhaumont/ectrans/tree/inv_trans-split_denis

- #287 Dump checksums in ectrans tests **MERGED**

  Bit reproducibility validation

- #317 Factorize duplicated between trltog and trgtol **MERGED**

  Code simplification

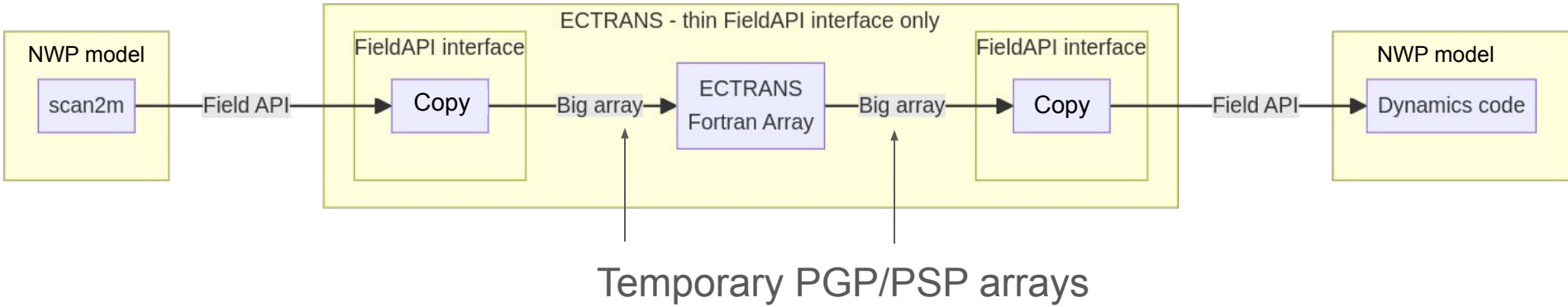
Thin Field API interface on top of ecTrans

Phase 1



MPL & ecTrans meeting - 03.11.2026
A Field API interface for ecTrans

INTERMEDIATE STEP AFTER PHASE 1



- ecTrans not modified
- NWP model can already be refactored
- drawback: extra memory copies to temporary arrays (GPU to GPU copies)

ADDING FIELD API INTERFACE

#314 Introduction of field API interface in ectrans **MERGED**

1. Allow any number of 2D/3D/4D vectors fields (U/V, VOR/DIV)
2. Allow any number of 2D/3D/4D scalar fields
3. Select GPU or CPU for computing and transfer data adequately
4. Avoid dummy fields (surface pressure)
5. Correctly manage FSPGL_PROC (abstract interface, null operator)

```
SUBROUTINE INV_TRANS_FIELD_API(KRESOL,                                    &
                             & LDACC,                                     &
                             & YDFSPSCALAR, YDFSPVOR,YDFSPDIV,            &
                             & YDFSCALAR, YDFU, YDFV,                     &
                             & YDFVOR,YDFDIV,                             &
                             & YDFSCALAR_NS, YDFSCALAR_EW, YDFU_EW, YDFV_EW, &
                             & FSPGL_PROC)


INTEGER(KIND=JPIM),    INTENT(IN), OPTIONAL  :: KRESOL
LOGICAL, INTENT(IN), OPTIONAL :: LDACC
TYPE(FIELD_SPEC),INTENT(IN), OPTIONAL  :: YDFSPSCALAR(:)                  ! SPECTRAL SCALAR FIELDS (IN)
TYPE(FIELD_SPEC),INTENT(IN), OPTIONAL  :: YDFSPVOR(:), YDFSPDIV(:)        ! SPECTRAL VECTOR FIELDS (IN)

TYPE(FIELD_GRID),INTENT(INOUT), OPTIONAL  :: YDFSCALAR(:)                    ! GRID SCALAR FIELDS     (OUT)
TYPE(FIELD_GRID),INTENT(INOUT), OPTIONAL  :: YDFU(:),YDFV(:)                 ! GRID VECTOR FIELDS     (OUT)
TYPE(FIELD_GRID),INTENT(INOUT), OPTIONAL  :: YDFVOR(:),YDFDIV(:)             ! GRID VECTOR FIELDS     (OUT)

TYPE(FIELD_GRID),INTENT(INOUT), OPTIONAL  :: YDFSCALAR_NS(:), YDFSCALAR_EW(:)    ! GRID SCALAR FIELDS DERIVATIVES EW
AND NS (OUT)
TYPE(FIELD_GRID),INTENT(INOUT), OPTIONAL  :: YDFU_EW(:),YDFV_EW(:)           ! GRID VECTOR FIELDS DERIVATIVES EW
(OUT)
```

LIST OF INPUT and OUTPUT FIELDS

```
TYPE FIELD_GRID
 ! A pointer to field API field with additional METADATA
 CLASS (FIELD_BASIC), POINTER :: PTR      ! POINTER TO FIELD_BASIC from field API
 CHARACTER(LEN=10)            :: NAME     ! Name
 INTEGER(KIND=JPIM)           :: DIM_V    ! Vertical dimension (0 if not applicable)
END TYPE

TYPE FIELD_SPEC
 ! A pointer to field API field with additional METADATA
 CLASS (FIELD_BASIC), POINTER    :: PTR      ! POINTER TO FIELD_BASIC from field API
 CHARACTER(LEN=10)               :: NAME     ! Name
 INTEGER(KIND=JPIM), ALLOCATABLE :: IVSET(:) ! b-set for spectral fields
END TYPE
```

Note: FIELD_BASIC are the Field API objects, encapsulated in FIELD_GRID and FIELD_SPEC

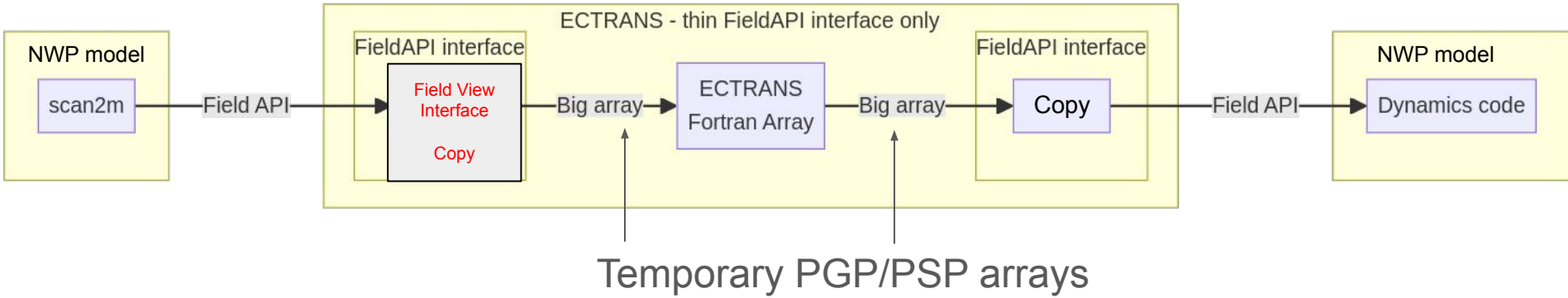
Introduction of intermediate field view interface

Phase 1b



MPL & ecTrans meeting - 03.11.2026
A Field API interface for ecTrans

A NEW INTERMEDIATE FIELD VIEW INTERFACE



[#441 Add a C pointer interface to ectrans](#) **MERGED**

- Field View interface is a new intermediate layer
- Portable C interface
- Will allow us to make a precision - and backend independent API
- Connection to Field API, Atlas, Python, etc

```
SUBROUTINE INV_TRANS_FIELD_VIEW(KRESOL, &
                              & YDSPSCALAR, YDSPVOR, YDSPDIV, &
                              & YDGPSCALAR, YDGPU, YDGPV,     &
                              & YDGPVOR,YDGPDIV, &
                              & YDGPSCALAR_NS, YDGPSCALAR_EW, YDGPU_EW, YDGPV_EW, &
                              & FSPGL_PROC)
! INPUT
INTEGER(KIND=INT32),INTENT(IN) :: KRESOL
TYPE(FIELD_VIEW),INTENT(IN)  :: YDSPSCALAR(:)                  ! SPECTRAL SCALAR FIELDS (IN)
TYPE(FIELD_VIEW),INTENT(IN)  :: YDSPVOR(:), YDSPDIV(:)        ! SPECTRAL VECTOR FIELDS (IN)

! OUTPUT (note that views have intent(IN), i.e. the view itself is immutable, not the data)
TYPE(FIELD_VIEW),INTENT(IN)  :: YDGPSCALAR(:)                   ! GRID SCALAR FIELDS  (OUT)
TYPE(FIELD_VIEW),INTENT(IN)  :: YDGPU(:),YDGPV(:)                ! GRID VECTOR FIELDS  (OUT)
TYPE(FIELD_VIEW),INTENT(IN)  :: YDGPVOR(:),YDGPDIV(:)            ! GRID VECTOR FIELDS (OUT)

TYPE(FIELD_VIEW),INTENT(IN)  :: YDGPSCALAR_EW(:), YDGPSCALAR_NS(:)  ! GRID SCALAR FIELDS DERIVATIVES EW
AND NS (OUT)
TYPE(FIELD_VIEW),INTENT(IN)  :: YDGPU_EW(:),YDGPV_EW(:)             ! GRID VECTOR FIELDS DERIVATIVES EW
(OUT)

PROCEDURE (FSPGL_INTF), POINTER, OPTIONAL, INTENT(IN)  :: FSPGL_PROC
```

```
! C-interoperable view of field data and its shape, stride, and dimension-role metadata.
! Array metadata uses C dimension ordering. Spectral fields may additionally reference
! an IVSET array assigning each level to a spectral distribution set.
TYPE, BIND(C) :: FIELD_VIEW

 ! ARRAY
 TYPE(C_PTR)     :: DATA
 INTEGER(C_INT) :: ACCESS
 INTEGER(C_INT) :: KIND
 INTEGER(C_INT) :: RANK
 INTEGER(C_INT) :: EXTENTS(MAX_RANK)  ! C ORDERING
 INTEGER(C_INT) :: STRIDES(MAX_RANK)  ! C ORDERING

 ! IVSET
 TYPE(C_PTR)     :: IVSET ! Value indexing is specified by IVSET_BASE
 INTEGER(C_INT) :: IVSET_SIZE
 INTEGER(C_INT) :: IVSET_DIM ! C ORDERING; -1 when IVSET does not index distribution array dimension
 INTEGER(C_INT) :: IVSET_BASE ! 0 for zero-based values, 1 for one-based values

 ! NAME
 TYPE(C_PTR)     :: NAME
 INTEGER(C_INT) :: NAME_SIZE

 ! INDEXING
 INTEGER(C_INT) :: INDEXING ! NOTE: C ORDERING

END TYPE
```

FIELD_VIEW contains a portable description of the fields, stored ad C_PTR. List of FIELD_VIEW are constructed from list of FIELD_GRID and FIELD_SPEC, by calling the GET_DEVICE_PTR functions

List of pointers in ecTrans

Phase 2



MPL & ecTrans meeting - 03.11.2026
A Field API interface for ecTrans

ecTrans also uses PGP/PSP internally

- same drawbacks as before

  (code is complicated)

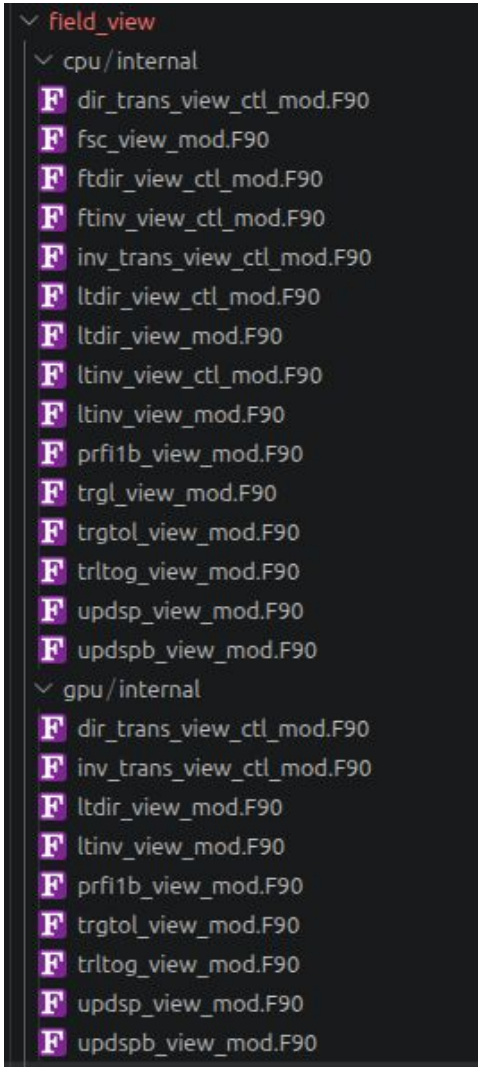
- replace PGVM/PGFL by **lists of pointers**

  ⇒ code greatly simplified

LIST OF POINTERS IN ECTRANS

#475 Propagation of pointers

- Pointers are propagated only to the top layers of ecTrans
  - before MPI calls
  - Internal routines are untouched
- Introduction of *_view routines
  - refactoring of the call tree of inv_trans and dir_trans to use lists of FIELD_VIEW
- GPU and CPU versions implemented
- *_view routines only used by from field view API
  - Do we want to use then in inv_trans and dir_trans ? (avoid code duplication)



POINTERS BEING PROPAGATED

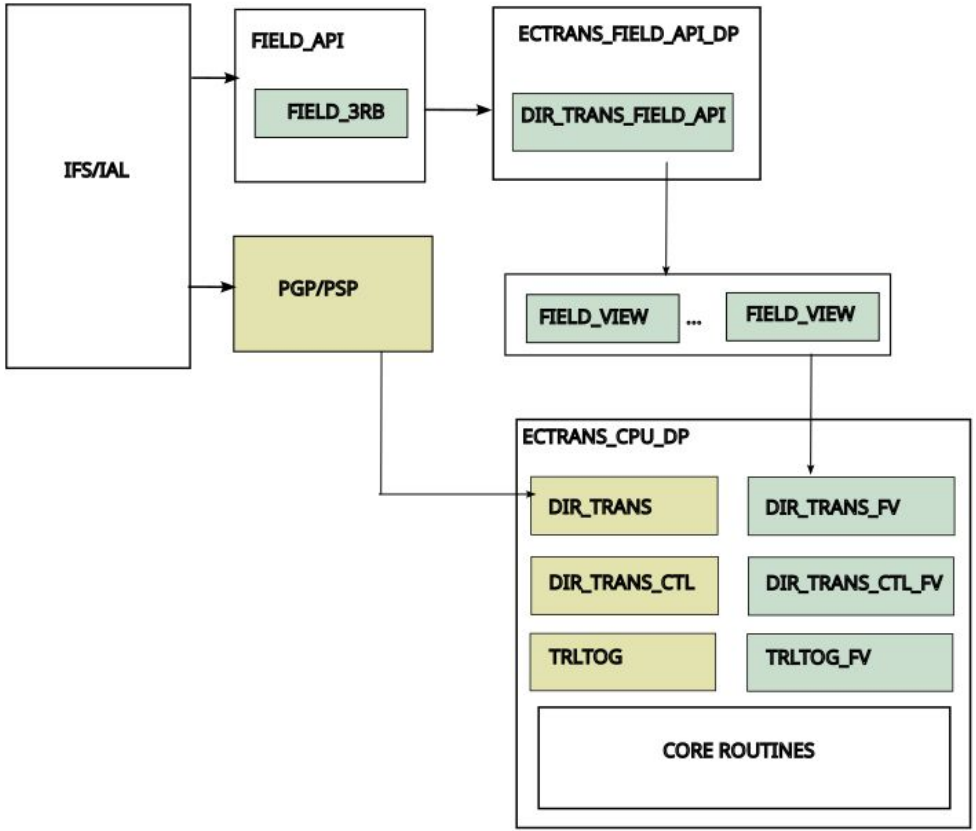
```
TYPE SPEC_VIEW
 ! Spectral field view
 REAL(KIND=JPRB),POINTER   :: P(:) => NULL()
 CHARACTER(LEN=:),POINTER  :: NAME => NULL()
END TYPE

TYPE GRID_VIEW
 ! Grid point field view
 REAL(KIND=JPRB),POINTER    :: P(:,:) => NULL()
 CHARACTER(LEN=:), POINTER  :: NAME   => NULL()
 INTEGER(KIND=JPIM)         :: IVSET
END TYPE
```

NOTE: SPEC_VIEW pointers are Fortran pointers, re-constructed from the C_PTR contained in the FIELD_VIEW objects

THE NEW ARCHITECTURE

- Introduction of new field view routines (FV)
- Field API interface calls Field View interface
- Field view interface can be interfaced with Atlas, Python, etc



RESULTS

- Results are bit-reproducible
- First performance tests in pair with reference version (inv_trans/dir_trans)
- More tests and validation is needed

PERFORMANCE EVALUATION

- Difficult to compare the reference and the field API versions
  - reference version performed Host to device synchronisation
  - Field API version runs entirely on the GPU
- LPGP_ON_GPU can avoid some host/device synchronisations when data already on GPU in the reference version

  (but LPSP_ON_GPU: not implemented yet)

- Previous issue on GPU - Solved by Nicolas Penigaud and Philippe Marguinaud
  - penalty hit due to a misuse of field API (a list of host memory pointers was used on device)
- Scalability test with ectrans-benchmark by Sam Hatfield in progress
    - high-level benchmarks on Grace Hopper

Field API interfaces in Arpege

Phase 3



MPL & ecTrans meeting - 03.11.2026
A Field API interface for ecTrans

USING ECTRANS FIELD API FROM ARPEGE

https://github.com/chesnelj/IAL/tree/50t2_ectransFieldApiForSpecrt

- Prototype of calling inv_trans_field_api from Arpege (CY50T2) by Jeremy Chesnel
- Working and bit-reproducible (CPU)
- Tests need to be performed on GPU
- Performance need to be evaluated (GPU and CPU)

Conclusions and remaining work

GLOBAL TRANSFORMS

- DONE
  - #314 Introduction of field API interface in ectrans
  - #441 Add a C pointer interface to ectrans
- IN PROGRESS
  - #475 Propagation of pointers: under review
  - Performance test
  - Test from Arpege (50t2_ectransFieldApiForSpecrt)
- REMAINING
  - allow to choose dynamically the backend (CPU or GPU, sp or dp, global or lam)
  - Implementation of NPROMATR
  - Implementation of Adjoint code path
  - Use *_view routines from INV_TRANS and DIR_TRANS
  - Test from IFS
  - Performance and validation tests needed

LOCAL TRANSFORMS

- Nothing has been done yet
- Waiting the validation and performance tests of global transform